Министерство образования Республики Беларусь

Учреждение образования «Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в ИС»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Яшков П.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
   - Реализовать вычисление градиента ŷ-exi для обновления весов.
2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:
   - На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
   - Сохранять значения Esp для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
   - MSE + фиксированный шаг (данные из ЛР №1);
   - MSE + адаптивный шаг (данные из ЛР №2);
   - BCE + фиксированный шаг;
   - BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x1,x2.
   - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE.

Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Вариант 12:

12.

x_1	x_2	e
6	2	0
-6	2	0
6	-2	1
-6	-2	0

Код программы:

```
import numpy as np
import matplotlib
matplotlib.rcParams['toolbar'] = 'None'
import matplotlib.pyplot as plt

X = np.array([
    [6, 2],
    [-6, 2],
    [6, -2],
    [-6, -2]
], dtype=float)

E = np.array([0, 0, 1, 0], dtype=float)
T_MSE = np.where(E == 0, -1, 1).astype(float)

X1_MIN, X1_MAX = -7.0, 7.0
X2_MIN, X2_MAX = -3.0, 3.0

Ee = 0.01
alpha_fixed = 0.01
max_epochs = 5000

def step(u):
    return 1 if u >= 0 else 0

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))
```

```
def net(x, w, b):
    return float(np.dot(w, x) + b)

def predict_mse_class(X, w, b):
    return np.array([step(net(x, w, b)) for x in X], dtype=int)

def predict_bce_prob(X, w, b):
    return np.array([sigmoid(net(x, w, b)) for x in X], dtype=float)

def predict_bce_class(X, w, b):
    probs = predict_bce_prob(X, w, b)
    return np.array([1 if p >= 0.5 else 0 for p in probs], dtype=int)

def bce_loss(y_true, y_pred):
    y_pred = np.clip(y_pred, 1e-12, 1 - 1e-12)
    return -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def train_mse_fixed(X, T, alpha=0.01, Ee=0.01, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, t in zip(X, T):
            y = net(x, w, b)
            err = t - y
            Es += err ** 2
            w += alpha * err * x
            b += alpha * err
        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break

    return w, b, np.array(Es_hist), epochs

def train_mse_adaptive(X, T, Ee=0.01, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, t in zip(X, T):
            y = net(x, w, b)
            err = t - y
            Es += err ** 2
            denom = np.dot(x, x) + 1.0
            alpha_t = 1.0 / denom if denom != 0 else 0.0
```

```
            w += alpha_t * err * x
            b += alpha_t * err
        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break

    return w, b, np.array(Es_hist), epochs

def train_bce_fixed(X, E, alpha=0.01, Ee=0.01, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, e in zip(X, E):
            z = net(x, w, b)
            y_hat = sigmoid(z)
            err = y_hat - e
            Es += bce_loss(e, y_hat)
            w -= alpha * err * x
            b -= alpha * err
        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break

    return w, b, np.array(Es_hist), epochs

def train_bce_adaptive(X, E, Ee=0.01, max_epochs=5000):
    w = np.zeros(X.shape[1], dtype=float)
    b = 0.0
    Es_hist = []
    epochs = 0

    while epochs < max_epochs:
        Es = 0.0
        for x, e in zip(X, E):
            z = net(x, w, b)
            y_hat = sigmoid(z)
            err = y_hat - e
            Es += bce_loss(e, y_hat)
            denom = np.dot(x, x) + 1.0
            alpha_t = 1.0 / denom if denom != 0 else 0.0
            w -= alpha_t * err * x
            b -= alpha_t * err
        Es_hist.append(Es)
        epochs += 1
        if Es <= Ee:
            break
```

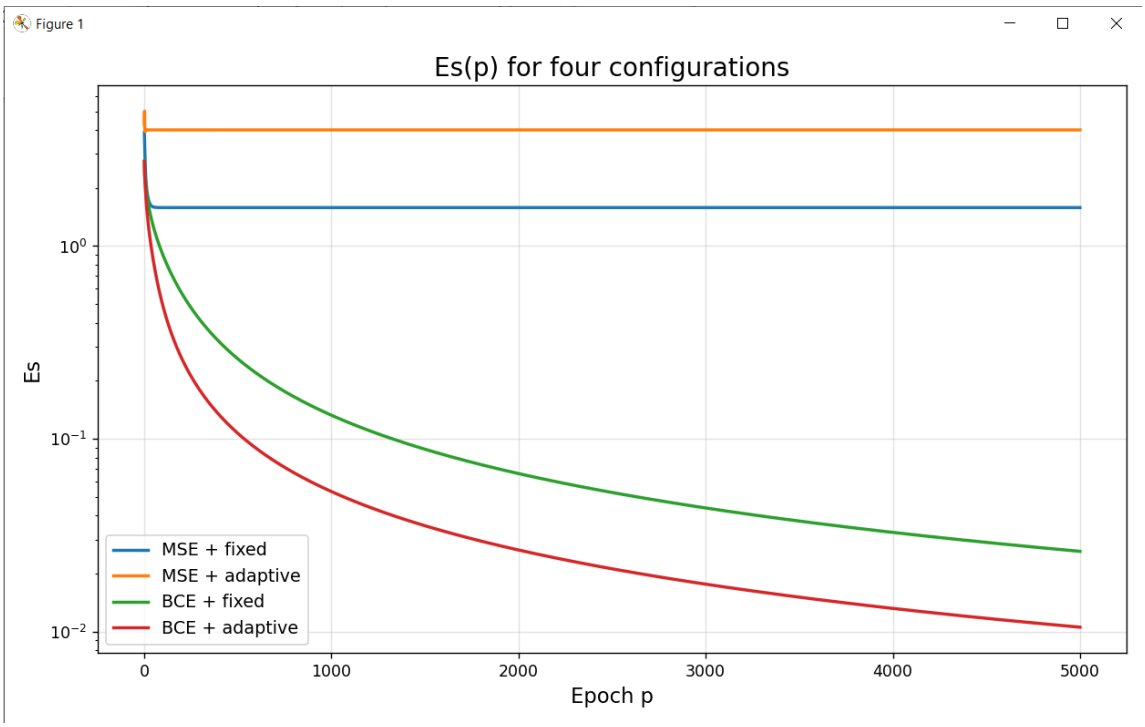
```
    return w, b, np.array(Es_hist), epochs

def plot_all_losses(hist_mse_fixed, hist_mse_adapt, hist_bce_fixed, hist_bce_adapt):
    plt.figure(figsize=(10, 6))

    hist_mse_fixed = np.clip(hist_mse_fixed, 1e-12, 1e6)
    hist_mse_adapt = np.clip(hist_mse_adapt, 1e-12, 1e6)
    hist_bce_fixed = np.clip(hist_bce_fixed, 1e-12, 1e6)
    hist_bce_adapt = np.clip(hist_bce_adapt, 1e-12, 1e6)

    plt.plot(np.arange(1, len(hist_mse_fixed) + 1), hist_mse_fixed, linewidth=2, label="MSE + fixed")
    plt.plot(np.arange(1, len(hist_mse_adapt) + 1), hist_mse_adapt, linewidth=2, label="MSE + adaptive")
    plt.plot(np.arange(1, len(hist_bce_fixed) + 1), hist_bce_fixed, linewidth=2, label="BCE + fixed")
    plt.plot(np.arange(1, len(hist_bce_adapt) + 1), hist_bce_adapt, linewidth=2, label="BCE + adaptive")

    plt.title("Es(p) for four configurations", fontsize=16)
    plt.xlabel("Epoch p", fontsize=13)
    plt.ylabel("Es", fontsize=13)
    plt.yscale("log")
    plt.grid(True, alpha=0.35)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.show()

def plot_lines_comparison(X, E, w_mse, b_mse, w_bce, b_bce, user_point=None):
    plt.figure(figsize=(10, 8))
    plt.xlabel("x1", fontsize=13)
    plt.ylabel("x2", fontsize=13)
    plt.title("Comparison of decision boundaries", fontsize=16)
    plt.grid(True, alpha=0.35)

    xx, yy = np.meshgrid(
        np.linspace(X1_MIN, X1_MAX, 300),
        np.linspace(X2_MIN, X2_MAX, 300)
    )
    zz = w_bce[0] * xx + w_bce[1] * yy + b_bce
    probs = sigmoid(zz)
    plt.contourf(xx, yy, probs, levels=[0.0, 0.5, 1.0], alpha=0.12)

    X0 = X[E == 0]
    X1c = X[E == 1]

    if len(X0) > 0:
        plt.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="class 0")
    if len(X1c) > 0:
        plt.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="class
1")

    x_vals = np.linspace(X1_MIN, X1_MAX, 300)

    if abs(w_mse[1]) >= 1e-12:
```

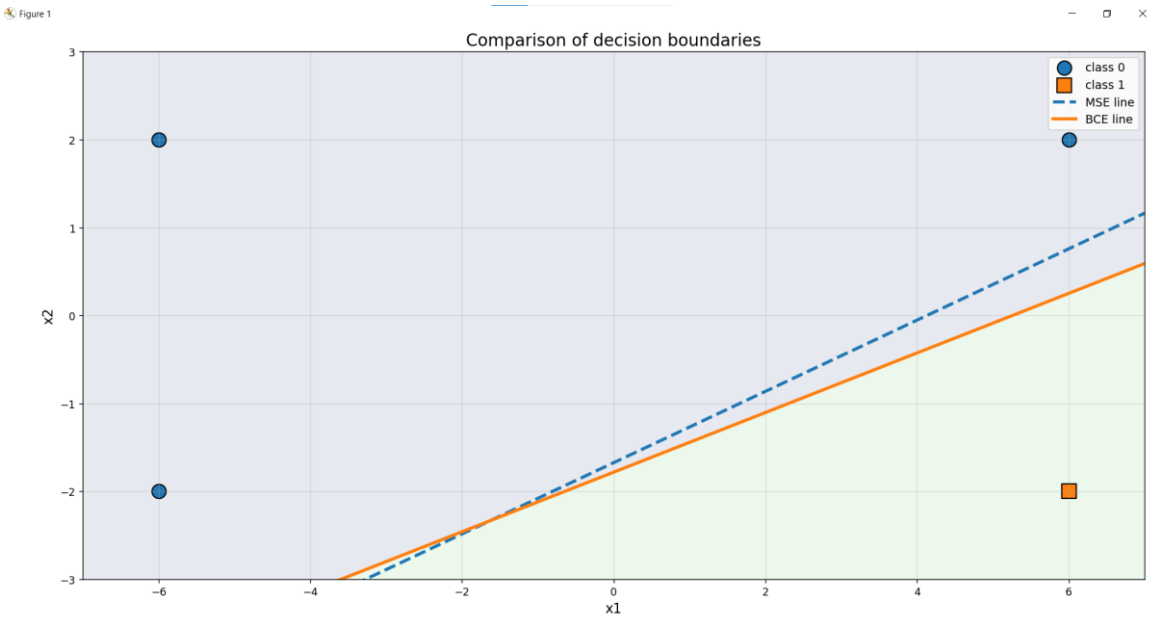
```
        y_vals_mse = -(w_mse[0] * x_vals + b_mse) / w_mse[1]
        plt.plot(x_vals, y_vals_mse, linewidth=3, linestyle="--", label="MSE line")
    elif abs(w_mse[0]) >= 1e-12:
        x_const = -b_mse / w_mse[0]
        plt.axvline(x=x_const, linewidth=3, linestyle="--", label="MSE line")

    if abs(w_bce[1]) >= 1e-12:
        y_vals_bce = -(w_bce[0] * x_vals + b_bce) / w_bce[1]
        plt.plot(x_vals, y_vals_bce, linewidth=3, label="BCE line")
    elif abs(w_bce[0]) >= 1e-12:
        x_const = -b_bce / w_bce[0]
        plt.axvline(x=x_const, linewidth=3, label="BCE line")

    if user_point is not None:
        x_user = np.array(user_point, dtype=float)
        prob = sigmoid(net(x_user, w_bce, b_bce))
        cls = 1 if prob >= 0.5 else 0
        marker = "*" if cls == 1 else "x"
        plt.scatter([x_user[0]], [x_user[1]], marker=marker, s=220, linewidths=2.2,
                    label=f"user: class={cls}, p={prob:.3f}")

    plt.xlim(X1_MIN, X1_MAX)
    plt.ylim(X2_MIN, X2_MAX)
    plt.legend(fontsize=11)
    plt.tight_layout()
    plt.show()

def interactive_mode_bce(w_bce, b_bce, w_mse, b_mse):
    plt.ion()
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.set_xlabel("x1", fontsize=13)
    ax.set_ylabel("x2", fontsize=13)
    ax.set_title("Interactive mode: BCE adaptive", fontsize=16)
    ax.grid(True, alpha=0.35)
    ax.set_xlim(X1_MIN, X1_MAX)
    ax.set_ylim(X2_MIN, X2_MAX)

    xx, yy = np.meshgrid(
        np.linspace(X1_MIN, X1_MAX, 300),
        np.linspace(X2_MIN, X2_MAX, 300)
    )
    zz = w_bce[0] * xx + w_bce[1] * yy + b_bce
    probs = sigmoid(zz)
    ax.contourf(xx, yy, probs, levels=[0.0, 0.5, 1.0], alpha=0.12)

    X0 = X[E == 0]
    X1c = X[E == 1]

    if len(X0) > 0:
        ax.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="class 0")
    if len(X1c) > 0:
        ax.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="class 1")
```

```
    x_vals = np.linspace(X1_MIN, X1_MAX, 300)

    if abs(w_mse[1]) >= 1e-12:
        y_vals_mse = -(w_mse[0] * x_vals + b_mse) / w_mse[1]
        ax.plot(x_vals, y_vals_mse, linewidth=3, linestyle="--", label="MSE line")
    elif abs(w_mse[0]) >= 1e-12:
        x_const = -b_mse / w_mse[0]
        ax.axvline(x=x_const, linewidth=3, linestyle="--", label="MSE line")

    if abs(w_bce[1]) >= 1e-12:
        y_vals_bce = -(w_bce[0] * x_vals + b_bce) / w_bce[1]
        ax.plot(x_vals, y_vals_bce, linewidth=3, label="BCE line")
    elif abs(w_bce[0]) >= 1e-12:
        x_const = -b_bce / w_bce[0]
        ax.axvline(x=x_const, linewidth=3, label="BCE line")

    ax.legend(fontsize=11)
    fig.canvas.draw()
    fig.canvas.flush_events()

    while True:
        s = input("x1 x2 (or q): ").strip()
        if s.lower() in ("q", "quit", "exit"):
            break
        try:
            parts = s.replace(",", " ").split()
            if len(parts) != 2:
                print("Enter 2 numbers")
                continue
            x1v = float(parts[0])
            x2v = float(parts[1])
        except:
            print("Input error")
            continue

        prob = sigmoid(net(np.array([x1v, x2v], dtype=float), w_bce, b_bce))
        cls = 1 if prob >= 0.5 else 0
        marker = "*" if cls == 1 else "x"
        ax.scatter([x1v], [x2v], marker=marker, s=220, linewidths=2.2)
        fig.canvas.draw()
        fig.canvas.flush_events()

        print(f"probability = {prob:.6f}, class = {cls}")

    plt.ioff()
    plt.show()

def main():
    w_mse_fixed, b_mse_fixed, Es_mse_fixed, ep_mse_fixed = train_mse_fixed(
        X, T_MSE, alpha=alpha_fixed, Ee=Ee, max_epochs=max_epochs
    )
```

```
    w_mse_adapt, b_mse_adapt, Es_mse_adapt, ep_mse_adapt = train_mse_adaptive(
        X, T_MSE, Ee=Ee, max_epochs=max_epochs
    )
    w_bce_fixed, b_bce_fixed, Es_bce_fixed, ep_bce_fixed = train_bce_fixed(
        X, E, alpha=alpha_fixed, Ee=Ee, max_epochs=max_epochs
    )
    w_bce_adapt, b_bce_adapt, Es_bce_adapt, ep_bce_adapt = train_bce_adaptive(
        X, E, Ee=Ee, max_epochs=max_epochs
    )

    pred_mse_fixed = predict_mse_class(X, w_mse_fixed, b_mse_fixed)
    pred_mse_adapt = predict_mse_class(X, w_mse_adapt, b_mse_adapt)
    pred_bce_fixed = predict_bce_class(X, w_bce_fixed, b_bce_fixed)
    pred_bce_adapt = predict_bce_class(X, w_bce_adapt, b_bce_adapt)

    acc_mse_fixed = np.mean(pred_mse_fixed == E.astype(int))
    acc_mse_adapt = np.mean(pred_mse_adapt == E.astype(int))
    acc_bce_fixed = np.mean(pred_bce_fixed == E.astype(int))
    acc_bce_adapt = np.mean(pred_bce_adapt == E.astype(int))

    print("Comparison of four configurations:")
    print(f"MSE + fixed:    epochs={ep_mse_fixed}, Es={Es_mse_fixed[-1]:.6e},
acc={acc_mse_fixed:.2f}")
    print(f"MSE + adaptive: epochs={ep_mse_adapt}, Es={Es_mse_adapt[-1]:.6e},
acc={acc_mse_adapt:.2f}")
    print(f"BCE + fixed:    epochs={ep_bce_fixed}, Es={Es_bce_fixed[-1]:.6e}, acc={acc_bce_fixed:.2f}")
    print(f"BCE + adaptive: epochs={ep_bce_adapt}, Es={Es_bce_adapt[-1]:.6e},
acc={acc_bce_adapt:.2f}")

    plot_all_losses(Es_mse_fixed, Es_mse_adapt, Es_bce_fixed, Es_bce_adapt)
    plot_lines_comparison(X, E, w_mse_fixed, b_mse_fixed, w_bce_adapt, b_bce_adapt)
    interactive_mode_bce(w_bce_adapt, b_bce_adapt, w_mse_fixed, b_mse_fixed)

if __name__ == "__main__":
    main()
```

Результат :





В ходе выполнения лабораторной работы была реализована бинарная классификация с использованием логистической регрессии и функции потерь бинарной кросс-энтропии (BCE). Для сравнения были рассмотрены четыре варианта обучения модели: MSE с фиксированным шагом (ЛР1), MSE с адаптивным шагом (ЛР2), BCE с фиксированным шагом и BCE с адаптивным шагом. Для всех методов были построены кривые сходимости ошибки Es(p) и выполнено сравнение количества эпох, необходимых для достижения заданной точности.

Результаты показали, что использование функции потерь BCE обеспечивает более стабильное и быстрое обучение по сравнению с MSE при решении задачи бинарной классификации. Это связано с тем, что BCE напрямую оптимизирует вероятность принадлежности объекта к классу и лучше подходит для задач классификации, тогда как MSE изначально предназначена для задач регрессии. Также было установлено, что применение адаптивного шага обучения ускоряет сходимость алгоритма по сравнению с фиксированным шагом, поскольку размер шага автоматически подстраивается под текущий входной вектор.

При сравнении разделяющих линий, полученных различными методами, было отмечено, что линии в целом имеют схожую ориентацию, однако модель с BCE формирует более корректную вероятностную границу классификации. Это позволяет точнее интерпретировать результат как вероятность принадлежности к классу.

Таким образом, для задач бинарной классификации более предпочтительным является использование функции потерь бинарной кросс-энтропии в сочетании с сигмоидной функцией активации. Дополнительное применение адаптивного шага обучения позволяет ускорить процесс обучения и повысить устойчивость алгоритма.

Вывод:

Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провел **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2)